Laboratory sessions

LAB RRT - Implemlentation

Exercice 1.1 (Install and test the Ray-Triangle intersection algorithm)

To speed up the development process and only focus on the RRT algorithm, we provide two additional classes:

1. A `Vec3` class is available with basic operators `Vec3.py`, as well as a `intersect_objects()` function which computes the closest intersection, given a ray in a `Vec3` format, to a list of objects (represented as meshes) and a list of Transforms (each transform is associated to a mesh).
2. Test the code on different simple examples to get used to the libraries

Exercice 1.2 (Building a single RRT)

Given a 3D scene, a starting point `s` and an ending point `e`, we now want to build a RRT from that will connect `s` and `e`.

1. start by building a data structure to represent the graph
2. implement the single RRT approach (following the course)
3. test your code on different simple scenes (cubes). Evaluate the performance of the code (eg total computation time and computation time and average computation time per node)

Exercice 1.3 (Building a double RRT)

Rely on the single RRT implementation to develop a double RRT implementation in which two graphs are built in parallel, one from the starting point `s` and one from the ending point `e`.

Exercice 1.4 (Searching a path)

Implement a search algorithm to find the path between the initial and final points (no need to implement an elaborate search here)

Exercice 1.5 (Smoothing the path)

Propose an algorithm to smooth the path between point `s` and point `e` while still **avoiding obstacles**.